UNIVERSIDADE DE FORTALEZA – UNIFOR
CENTRO DE CIÊNCIAS TECNOLÓGICAS
CURSO DE CIÊNCIA DA COMPUTAÇÃO

COMPUTAÇÃO PARALELA CONCORRENTE

LUCAS DE VASCONCELOS BARREIRA CARVALHO

ANÁLISE DE DESEMPENHO DE ALGORITMOS DE ORDENAÇÃO EM AMBIENTES CONCORRENTES E PARALELOS: UM ESTUDO COMPARATIVO EM JAVA

Relatório de trabalho prático apresentado à disciplina Computação Paralela Concorrente, do curso de Ciência da Computação da Universidade de Fortaleza, como requisito parcial de avaliação.

Aluno: Lucas de Vasconcelos Barreira Carvalho – Matrícula 2410424

Professor: Izequiel Pereira de Norões

FORTALEZA
2026

SUMÁRIO

RESUMO.....**3**
1 INTRODUÇÃO.....**4**
1.1 Algoritmos escolhidos.....4
1.2 Estratégias de paralelização.....5
1.3 Métricas de desempenho.....5
2 METODOLOGIA.....**7**
2.1 Ambiente de execução.....7
2.2 Framework de teste.....7
2.3 Conjuntos de dados de entrada.....8
2.4 Configurações de processamento.....8
2.5 Arquivos CSV.....8
2.6 Análise estatística.....9
2.7 Visualização dos resultados.....9
3 RESULTADOS E DISCUSSÃO.....**11**
3.1 Comparação entre as versões seriais.....11
3.2 Versões serial e paralela de cada algoritmo.....12
3.3 Efeito do tamanho da entrada.....13
3.4 Efeito da natureza dos dados.....14
3.5 Efeito do número de núcleos.....17
3.6 Variabilidade das medições.....19
3.7 Síntese: qual algoritmo para qual cenário.....20
4 CONCLUSÃO.....**21**
REFERÊNCIAS.....**22**
ANEXO A – REPOSITÓRIO DO CÓDIGO.....**23**
ANEXO B – CÓDIGO-FONTE DAS IMPLEMENTAÇÕES.....**24**

RESUMO

Este trabalho analisa o desempenho de quatro algoritmos de ordenação – Bubble Sort, Quick Sort, Merge Sort e Counting Sort – em versões serial e paralela, implementadas em Java 8 com as APIs de concorrência da plataforma (Fork/Join e ExecutorService). Foi desenvolvido um framework de teste que gera entradas reproduzíveis de quatro naturezas (aleatória, ordenada, inversa e com poucos valores distintos) e de tamanhos entre mil e cinco milhões de elementos, executa cada configuração (serial e paralela com 1, 2, 4 e 8 threads) em cinco amostras, confere a correção de cada resultado e grava os tempos em arquivos CSV. Um segundo experimento limitou a JVM a um e a dois núcleos. Ao todo foram registradas 3.600 medições, analisadas por média, desvio padrão, coeficiente de variação, intervalo de confiança de 95%, speedup e eficiência, e visualizadas em gráficos estáticos e em um visualizador dinâmico em Swing. Com dois núcleos, Merge Sort e Quick Sort paralelos atingiram speedup de 1,94 e 1,85 para cinco milhões de elementos, próximo do limite de 2; abaixo de cerca de 50 mil elementos, o custo de criar e coordenar threads superou o ganho. O Bubble Sort paralelo, baseado em particionamento em blocos, chegou a speedup de 24,7 com 8 threads, valor superlinear explicado pela redução do trabalho total (de n^2 para n^2/p) e não apenas pelo paralelismo, como confirmou o experimento com um único núcleo. O Counting Sort, limitado pela largura de banda de memória e pela amplitude dos valores, só obteve ganho (até 1,49) para entradas muito grandes. Conclui-se que o paralelismo compensa apenas quando o trabalho por thread é grande em relação ao custo de coordenação e que aumentar o número de threads além do número de núcleos físicos não traz ganho.

Palavras-chave: algoritmos de ordenação; programação paralela; Java; Fork/Join; speedup; análise de desempenho.

1 INTRODUÇÃO

A ordenação é uma das operações mais frequentes em computação: aparece em bancos de dados, em mecanismos de busca, na compressão de dados e como etapa de inúmeros outros algoritmos. Com a popularização dos processadores multinúcleo, deixou de bastar escolher o algoritmo de menor complexidade assintótica; passa a importar também o quanto do seu trabalho pode ser dividido entre núcleos e qual é o custo de coordenar essa divisão.

O objetivo deste trabalho é comparar, de forma experimental, o desempenho de algoritmos de ordenação em suas versões serial e paralela, variando o tamanho e a natureza dos dados de entrada, o número de threads e o número de núcleos de processamento disponíveis. Embora o enunciado se refira a "algoritmos de busca", todos os métodos listados são algoritmos de ordenação, e é assim que são tratados neste texto.

A abordagem adotada foi: (i) implementar cada algoritmo em Java com uma versão serial e uma paralela sob a mesma interface; (ii) construir um framework de teste que gera os dados, executa e cronometra todas as configurações, verifica os resultados e grava os tempos em CSV; (iii) calcular as estatísticas e as métricas de desempenho a partir dos CSV; e (iv) apresentar os resultados em gráficos, incluindo um visualizador dinâmico em Swing.

1.1 Algoritmos escolhidos

Dos seis algoritmos sugeridos, foram escolhidos quatro que representam famílias de comportamento diferentes, o que torna a comparação mais informativa:

- **Bubble Sort** – algoritmo comparativo de custo $O(n^2)$, com parada antecipada quando uma passada não faz trocas (custo $O(n)$ para entradas já ordenadas). Representa os algoritmos quadráticos simples.
- **Quick Sort** – algoritmo de divisão e conquista, $O(n \log n)$ no caso médio. Foi usado pivô pela mediana de três e particionamento de Hoare, o que evita o pior caso $O(n^2)$ em entradas ordenadas ou invertidas e mantém as partições equilibradas com muitos valores repetidos. Subvetores de até 32 elementos são finalizados com Insertion Sort.
- **Merge Sort** – divisão e conquista com custo $O(n \log n)$ em qualquer caso e memória auxiliar $O(n)$. A intercalação é pulada quando as duas metades já estão em ordem, e subvetores pequenos também usam Insertion Sort.
- **Counting Sort** – algoritmo não comparativo, $O(n + k)$, em que k é a amplitude dos valores (máximo − mínimo + 1). Seu desempenho depende mais do intervalo de valores do que da ordem dos elementos.

1.2 Estratégias de paralelização

Cada algoritmo implementa a interface `AlgoritmoOrdenacao`, com os métodos `ordenarSerial(int[])` e `ordenarParalelo(int[], int numThreads)`. A criação e o encerramento do pool de threads fazem parte do tempo medido da versão paralela, pois são um custo real de quem decide paralelizar. As estratégias foram:

- **Quick Sort e Merge Sort – Fork/Join.** Os dois são naturalmente recursivos: após particionar (Quick) ou dividir ao meio (Merge), as duas metades são ordenadas por tarefas independentes (`RecursiveAction`) em um `ForkJoinPool` com o número de threads desejado. Abaixo de 8.192 elementos, a tarefa resolve o restante de forma serial, para que o custo de criar tarefas não domine. No Merge Sort, cada intercalação é feita pela tarefa pai de forma serial, de modo que a intercalação final, de n elementos, é uma parte sequencial do algoritmo.
- **Bubble Sort – particionamento em blocos.** O vetor é dividido em p blocos contíguos; cada thread ordena o seu bloco com Bubble Sort e, em seguida, os blocos são intercalados em árvore ($\log_2 p$ níveis), com os pares de cada nível intercalados em paralelo. É a forma usual de paralelizar algoritmos quadráticos e tem uma consequência importante: o trabalho total cai de n^2 para $p \cdot (n/p)^2 = n^2/p$, ainda que haja apenas um núcleo.
- **Counting Sort – quatro fases de dados paralelos.** Com um `ExecutorService` de p threads: (1) mínimo e máximo de cada bloco; (2) histograma local de cada bloco, sem disputa entre threads; (3) soma dos histogramas, com a faixa de valores dividida entre as threads; (4) reescrita do vetor, em que cada thread escreve os valores da sua faixa a partir de uma posição obtida por soma de prefixos. Não há travas nem variáveis atômicas: cada thread escreve apenas na sua própria região de memória.

Em todos os casos, a sincronização entre fases é feita aguardando os `Future` das tarefas (ou por `invokeAll`, no Fork/Join), o que estabelece a relação *happens-before* exigida pelo modelo de memória do Java e garante que cada fase veja os dados escritos pela anterior.

1.3 Métricas de desempenho

Para cada configuração, o **speedup** é a razão entre o tempo médio da versão serial e o tempo médio da versão paralela, S(p) = T_serial / T_paralelo(p), sempre comparando o mesmo algoritmo, distribuição, tamanho e quantidade de núcleos. A **eficiência** é E(p) = S(p) / p. Speedup maior que 1 indica ganho; o valor ideal é p quando há p núcleos livres. Pela Lei de Amdahl, se uma fração f do trabalho é

sequencial, o speedup máximo é 1 / (f + (1 − f)/p), e, em uma máquina com c núcleos, threads além de c não aumentam a capacidade de processamento, apenas disputam os mesmos núcleos.

2 METODOLOGIA

A metodologia seguiu o roteiro do enunciado: implementação das versões serial e paralela, desenvolvimento de um framework de teste, execução em diferentes configurações de processamento, registro dos tempos em CSV e análise estatística dos resultados para identificar padrões de desempenho.

2.1 Ambiente de execução

Tabela 1 – Ambiente de execução dos experimentos

Item	Valor
Processador	Intel Xeon @ 2,10 GHz (máquina virtual)
Núcleos disponíveis	2 (1 thread por núcleo)
Cache	L2 de 2 MiB por núcleo; L3 compartilhada
Memória	7,8 GiB (heap da JVM limitado a 3 GiB)
Sistema operacional	Linux 6.18 (64 bits)
JVM	OpenJDK 21.0.12 (código compilado para Java 8)
Data das execuções	29/10/2026

Fonte: elaborado pelo autor a partir dos arquivos CSV do experimento.

O ambiente tem apenas dois núcleos, o que limita o speedup ideal das versões paralelas a 2 e permite observar diretamente o efeito de usar mais threads do que núcleos (4 e 8 threads). Por ser uma máquina virtual compartilhada, há mais ruído de medição do que em um computador dedicado, o que é tratado na seção 3.6. Os scripts do projeto permitem repetir todo o experimento em qualquer máquina com Java 8 ou superior.

2.2 Framework de teste

O framework (classe `Benchmark`) recebe por parâmetros os algoritmos, os tamanhos, as distribuições, as quantidades de threads, o número de amostras e os arquivos de saída. Seu funcionamento é o seguinte:

1. **Aquecimento da JVM.** Antes das medições, cada algoritmo é executado cinco vezes em todas as configurações para que o compilador JIT otimize o código; sem isso, as primeiras medições seriam dominadas por código interpretado.
2. **Geração dos dados.** Para cada algoritmo, tamanho, distribuição e amostra, um vetor é gerado a partir de uma semente fixa. Todas as configurações (serial, P1, P2, P4 e P8) da mesma amostra recebem cópias do mesmo vetor, o que torna a comparação justa.
3. **Cronometragem.** Antes de cada execução, `System.gc()` é chamado para reduzir a chance de uma coleta de lixo cair dentro da medição. O tempo é medido com `System.nanoTime()` imediatamente antes e depois da chamada de ordenação.

4. **Verificação.** Fora do trecho cronometrado, o vetor resultante é comparado com uma ordenação de referência (`Arrays.sort`). O resultado é gravado na coluna `ordenado_ok`; todas as 3.600 execuções foram corretas.
5. **Registro.** Cada execução vira uma linha do CSV de resultados brutos, gravada e descarregada imediatamente, o que permite acompanhar o benchmark em tempo real pelo visualizador dinâmico.
6. **Resumo.** Ao final, a classe `Estatisticas` agrupa as amostras e gera o CSV de resumo.

2.3 Conjuntos de dados de entrada

Foram usadas quatro naturezas de entrada (enum `Distribuicao`), que variam tanto a ordem dos elementos quanto o intervalo de valores:

- **Aleatório:** valores uniformes no intervalo [0, 1.000.000).
- **Ordenado:** os mesmos valores do caso aleatório, em ordem crescente.
- **Inverso:** os mesmos valores, em ordem decrescente.
- **Poucos distintos:** valores uniformes em [0, 100), isto é, muitas repetições e amplitude pequena.

Os tamanhos foram 1.000, 10.000, 50.000, 100.000, 500.000, 1.000.000, 2.000.000 e 5.000.000 elementos para Quick Sort, Merge Sort e Counting Sort. Para o Bubble Sort, cujo custo quadrático tornaria inviáveis os tamanhos maiores (a extrapolação de 4,2 s para 50 mil elementos dá cerca de 12 horas para 5 milhões em uma única execução), usaram-se 1.000, 5.000, 10.000, 25.000 e 50.000 elementos.

2.4 Configurações de processamento

Cada combinação de algoritmo, tamanho e distribuição foi executada em cinco configurações: serial e paralela com 1, 2, 4 e 8 threads, com cinco amostras cada (2.900 medições na execução principal). A configuração paralela com 1 thread isola o custo da infraestrutura de paralelismo, já que não há paralelismo real.

Para investigar o efeito do número de núcleos, um segundo experimento repetiu um subconjunto (entradas aleatória e com poucos distintos; 25 mil elementos para o Bubble Sort e 1 e 5 milhões para os demais) com a JVM restrita a um núcleo e a dois núcleos. A restrição foi feita de duas formas combinadas: `taskset -c 0` fixa o processo na CPU 0 no sistema operacional, e a opção `-XX:ActiveProcessorCount=1` informa à JVM quantos processadores considerar (700 medições).

2.5 Arquivos CSV

Os resultados são gravados em três tipos de arquivo, na pasta `resultados`:

- `brutos.csv` (e `nucleos_N_brutos.csv`): uma linha por execução, com as colunas algoritmo, versao, threads, nucleos, tamanho, distribuicao, amostra, tempo_ms e ordenado_ok.
- `resumo.csv` (e `nucleos_resumo.csv`): uma linha por configuração, com amostras, média, desvio padrão, mínimo, máximo, mediana, coeficiente de variação, intervalo de confiança de 95%, speedup e eficiência.
- `log_execucao_principal.txt` e `log_experimento_nucleos.txt`: registro do andamento das execuções.

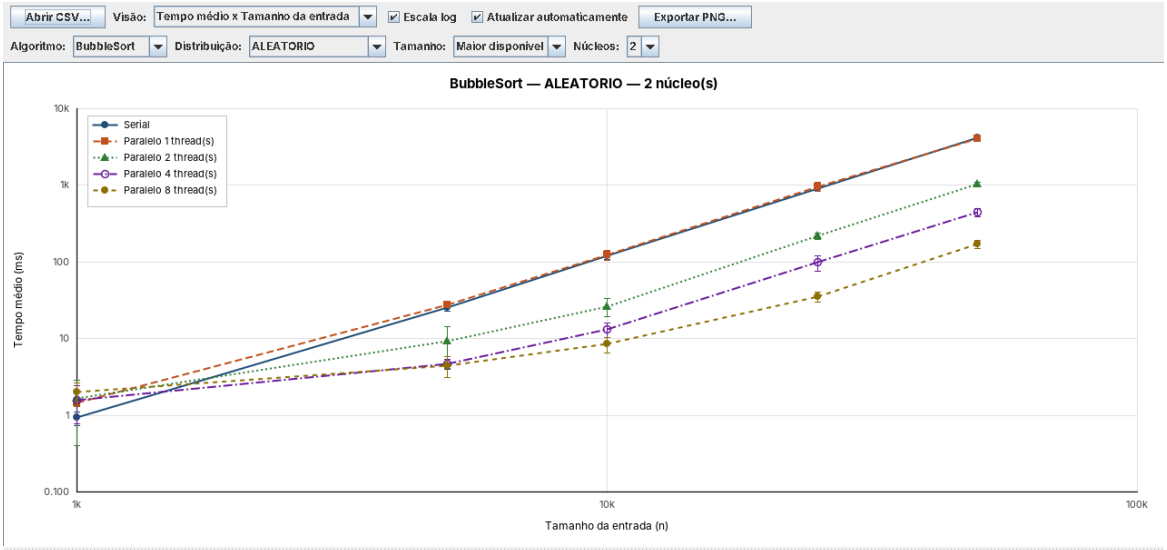
2.6 Análise estatística

Para cada grupo de cinco amostras foram calculados a média, o desvio padrão amostral (divisor n − 1), a mediana, o mínimo e o máximo, o coeficiente de variação (CV = desvio / média) e a meia-largura do intervalo de confiança de 95% da média, IC = t · s / √n, com t = 2,776 (distribuição t de Student com 4 graus de liberdade), conforme recomendado para avaliação de desempenho em Java por Georges, Buytaert e Eeckhout (2007). O IC é exibido como barra de erro nos gráficos: quando as barras de duas configurações se sobrepõem bastante, a diferença entre elas não deve ser considerada significativa. Speedup e eficiência foram calculados como descrito na seção 1.3. Todo esse processamento é feito em Java (classe `Estatisticas`), e pode ser refeito para qualquer conjunto de CSV.

2.7 Visualização dos resultados

Os gráficos deste relatório foram gerados pelo script `analise/gerar_graficos.py` (matplotlib), a partir dos mesmos CSV. Como ponto extra, foi desenvolvido o **visualizador de gráficos dinâmicos** `VisualizadorGraficos`, em Swing e sem bibliotecas externas. Ele lê os CSV brutos, calcula as estatísticas em memória e permite alternar entre seis visões (tempo × tamanho, speedup × threads, eficiência × threads, comparação entre algoritmos, efeito da distribuição e speedup por quantidade de núcleos), filtrando por algoritmo, distribuição, tamanho e núcleos. Os gráficos têm escala logarítmica opcional, barras de IC 95%, tooltip com o valor de cada ponto, tabela com os números da seleção e exportação para PNG. Com a opção "Atualizar automaticamente", o arquivo é relido a cada dois segundos, de modo que os gráficos acompanham em tempo real um benchmark em execução (Figuras 1 e 2).

Figura 1 – Visualizador dinâmico (Swing): tempo médio por tamanho da entrada para cada configuração

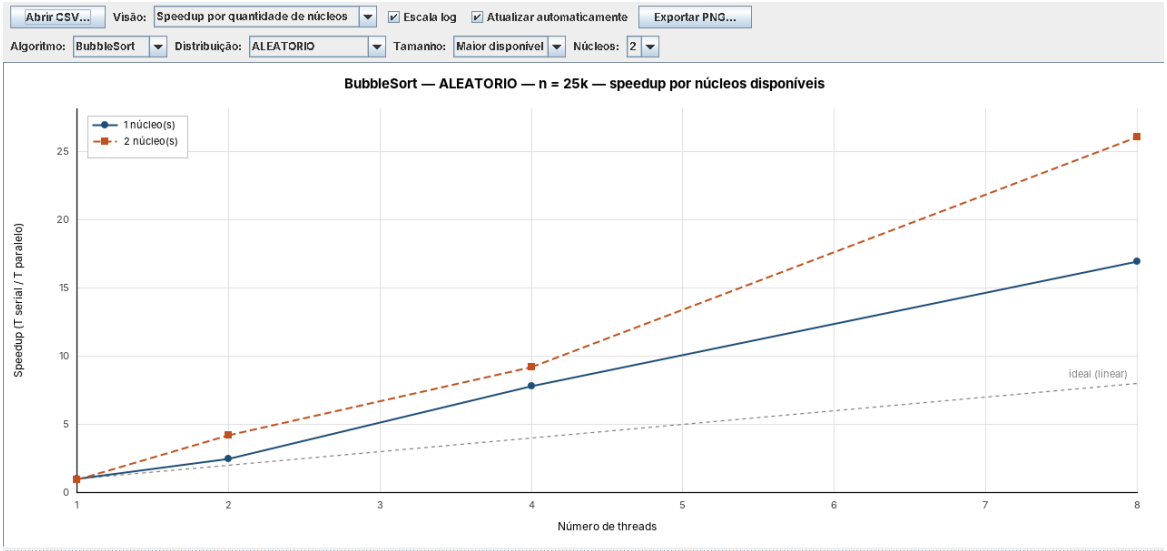


Núcleos	Algoritmo	Distribuição	Tamanho	Config.	Amostras	Média (ms)	Desvio (ms)	IC95 (ms)	CV (%)	Speedup	Eficiência
2	BubbleSort	ALEATORIO	1000	SERIAL	5	0.924	0.149	0.185	16.1	1.00	1.00
2	BubbleSort	ALEATORIO	1000	P1	5	1.456	0.138	0.171	9.5	0.63	0.63
2	BubbleSort	ALEATORIO	1000	P2	5	1.650	1.005	1.247	60.9	0.56	0.28
2	BubbleSort	ALEATORIO	1000	P4	5	1.590	0.663	0.823	41.7	0.58	0.15
2	BubbleSort	ALEATORIO	1000	P8	5	1.991	0.517	0.641	26.0	0.46	0.06
2	BubbleSort	ALEATORIO	5000	SERIAL	5	25.615	2.418	3.002	9.4	1.00	1.00
2	BubbleSort	ALEATORIO	5000	P1	5	27.913	0.844	1.048	3.0	0.92	0.92
2	BubbleSort	ALEATORIO	5000	P2	5	9.158	4.125	5.121	45.0	2.80	1.40
2	BubbleSort	ALEATORIO	5000	P4	5	4.678	0.541	0.672	11.6	5.48	1.37
2	BubbleSort	ALEATORIO	5000	P8	5	4.487	1.080	1.340	24.1	5.71	0.71
2	BubbleSort	ALEATORIO	10000	SERIAL	5	120.312	10.369	12.873	8.6	1.00	1.00
2	BubbleSort	ALEATORIO	10000	P1	5	122.498	14.621	18.152	11.9	0.98	0.98
2	BubbleSort	ALEATORIO	10000	P2	5	26.438	5.796	7.196	21.9	4.55	2.28

3250 medições carregadas de [resultados/brutos.csv, resultados/nucleos_1_brutos.csv]

Fonte: elaborado pelo autor a partir dos arquivos CSV do experimento.

Figura 2 – Visualizador dinâmico (Swing): speedup do Bubble Sort com 1 e 2 núcleos



Núcleos	Algoritmo	Distribuição	Tamanho	Config.	Amostras	Média (ms)	Desvio (ms)	IC95 (ms)	CV (%)	Speedup	Eficiência
1	BubbleSort	ALEATORIO	25000	P1	5	968.506	114.457	142.094	11.8	1.01	1.01
1	BubbleSort	ALEATORIO	25000	P2	5	401.338	47.062	58.426	11.7	2.44	1.22
1	BubbleSort	ALEATORIO	25000	P4	5	125.512	8.519	10.576	6.8	7.79	1.95
1	BubbleSort	ALEATORIO	25000	P8	5	57.755	7.269	9.024	12.6	16.93	2.12
2	BubbleSort	ALEATORIO	25000	P1	5	959.491	107.014	132.854	11.2	0.95	0.95
2	BubbleSort	ALEATORIO	25000	P2	5	218.160	18.470	22.929	8.5	4.18	2.09
2	BubbleSort	ALEATORIO	25000	P4	5	98.874	17.916	22.242	18.1	9.22	2.31
2	BubbleSort	ALEATORIO	25000	P8	5	34.974	4.245	5.270	12.1	26.08	3.26

3250 medições carregadas de [resultados/brutos.csv, resultados/nucleos_1_brutos.csv]

Fonte: elaborado pelo autor a partir dos arquivos CSV do experimento.

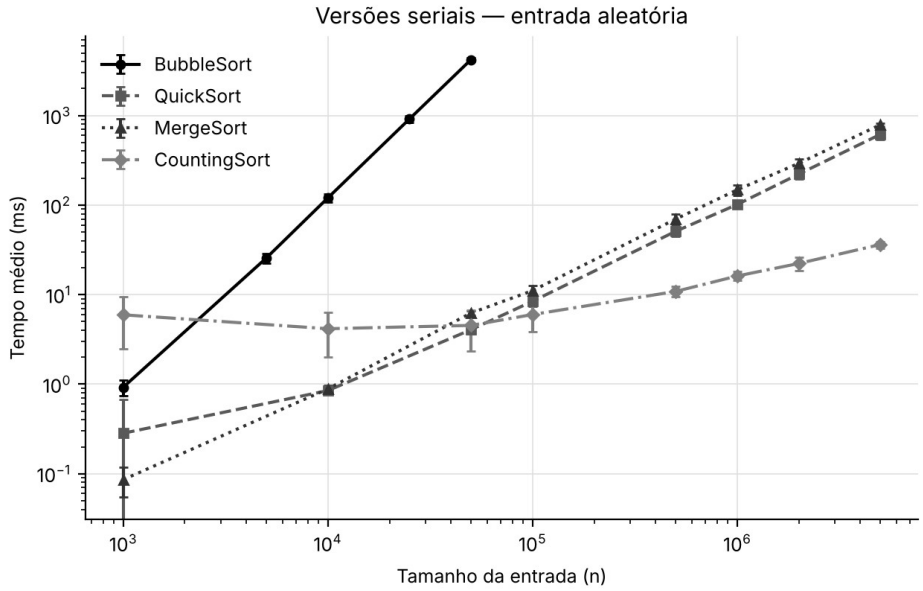
3 RESULTADOS E DISCUSSÃO

Esta seção apresenta os resultados das 3.600 medições. Os valores de tempo são médias de cinco amostras, acompanhadas da meia-largura do intervalo de confiança de 95%. Os valores completos estão em `resultados/resumo.csv` e podem ser explorados no visualizador dinâmico.

3.1 Comparação entre as versões seriais

A Figura 3 compara as versões seriais na entrada aleatória, em escala log-log. Nessa escala, a inclinação da reta indica a ordem de crescimento: a do Bubble Sort é cerca de 2 (quadrática), enquanto as do Quick Sort e do Merge Sort ficam próximas de 1 (n log n). A diferença prática é enorme: para 50 mil elementos, o Bubble Sort levou 4,2 s, contra 4,1 ms do Quick Sort, cerca de mil vezes menos.

Figura 3 – Tempo médio das versões seriais em função do tamanho, entrada aleatória (escalas logarítmicas)



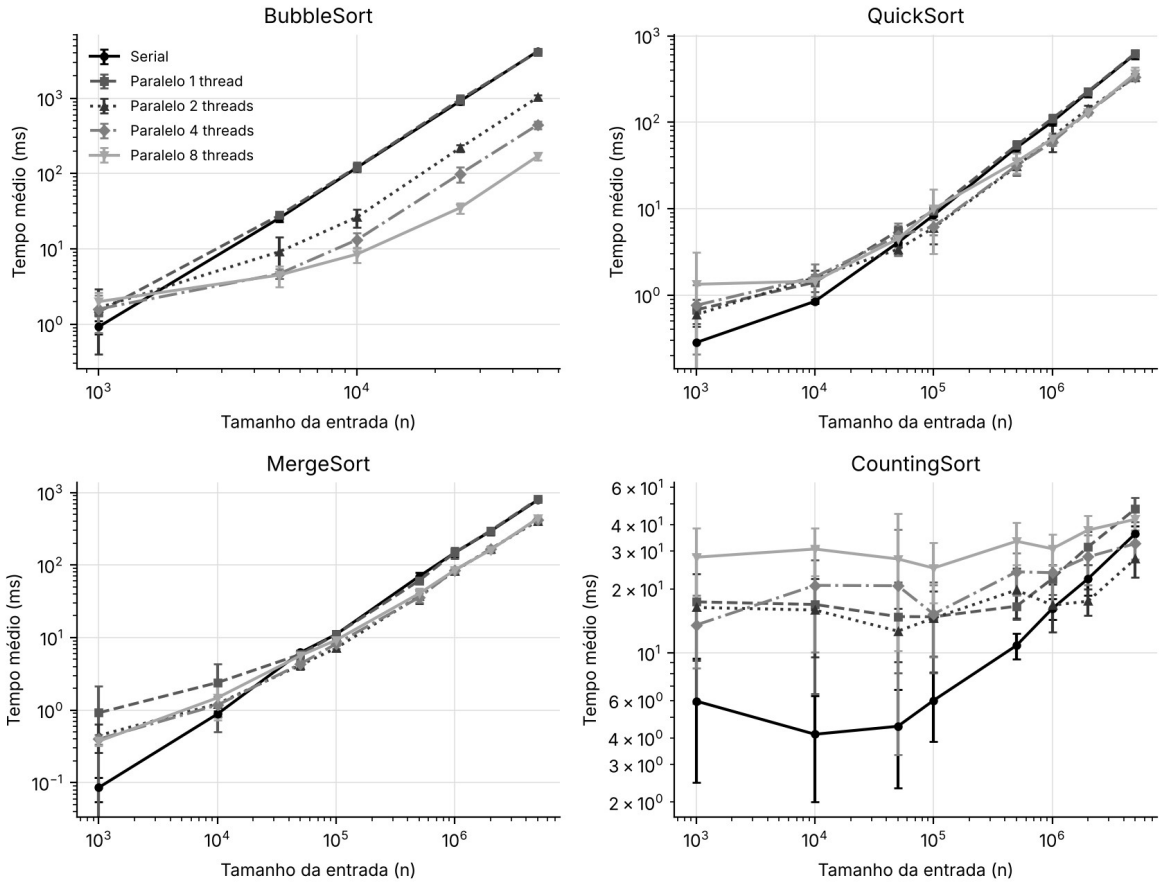
Fonte: elaborado pelo autor a partir dos arquivos CSV do experimento.

O Counting Sort tem comportamento peculiar. Para entradas pequenas ele é o mais lento entre os três algoritmos eficientes (5,95 ms para mil elementos, contra 0,09 ms do Merge Sort), porque o custo é dominado pela amplitude $k \approx 1.000.000$: é preciso alocar, zerar e percorrer um vetor de contagem de 4 MB, qualquer que seja n. À medida que n cresce, esse custo fixo se dilui, e para 5 milhões de elementos o Counting Sort foi o mais rápido, com 36 ms, contra 615 ms do Quick Sort e 788 ms do Merge Sort. É uma ilustração direta de que a complexidade O(n + k) depende do intervalo de valores, e não só do tamanho da entrada.

3.2 Versões serial e paralela de cada algoritmo

A Figura 4 mostra, para cada algoritmo, o tempo das cinco configurações em função do tamanho, e a Tabela 2 resume os tempos no maior tamanho testado com entrada aleatória.

Figura 4 – Tempo médio por configuração em função do tamanho, entrada aleatória (escalas logarítmicas)



Fonte: elaborado pelo autor a partir dos arquivos CSV do experimento.

Tabela 2 – Tempo médio (ms) ± IC 95% no maior tamanho, entrada aleatória, 2 núcleos

Algoritmo	n	Serial	P1	P2	P4	P8	Melhor speedup
BubbleSort	50.000	4.203,68 ± 194,70	4.118,65 ± 99,58	1.029,32 ± 81,73	443,78 ± 52,61	170,30 ± 19,10	24,68 (P8)
QuickSort	5.000.000	615,18 ± 74,56	627,54 ± 49,48	333,00 ± 35,73	334,00 ± 33,52	364,59 ± 67,92	1,85 (P2)
MergeSort	5.000.000	787,98 ± 28,49	806,10 ± 39,25	405,47 ± 28,00	416,97 ± 25,43	439,67 ± 27,98	1,94 (P2)
CountingSort	5.000.000	36,23 ± 3,13	47,33 ± 6,15	27,63 ± 4,98	32,58 ± 5,06	42,36 ± 6,70	1,31 (P2)

Fonte: elaborado pelo autor a partir dos arquivos CSV do experimento.

Quick Sort e Merge Sort. Para entradas grandes, as versões paralelas com 2 threads reduziram o tempo quase à metade: speedup de 1,85 no Quick Sort e 1,94 no Merge Sort com 5 milhões de elementos, ou seja, eficiência de 93% e 97%. É o

comportamento esperado de algoritmos de divisão e conquista, cujas metades são independentes. Com 4 e 8 threads o tempo não melhorou e, em alguns casos, piorou levemente (Quick Sort com 8 threads: 365 ms contra 333 ms com 2), porque a máquina tem apenas dois núcleos: as threads extras apenas se revezam nos mesmos núcleos, acrescentando trocas de contexto e tarefas a coordenar. A configuração P1 ficou muito próxima da serial (cerca de 2% mais lenta, dentro do IC), o que mostra que o custo fixo do Fork/Join é pequeno para entradas grandes.

O Merge Sort se aproximou mais do ideal do que o Quick Sort. No Merge Sort, a divisão é sempre exatamente ao meio, de modo que as duas threads recebem a mesma quantidade de trabalho; no Quick Sort, o tamanho das partições depende do pivô, e o primeiro particionamento – sobre o vetor inteiro – é feito por uma única thread, o que constitui uma fração sequencial maior. A intercalação final do Merge Sort também é sequencial, mas custa O(n), uma fração pequena do total O(n log n) para n grande.

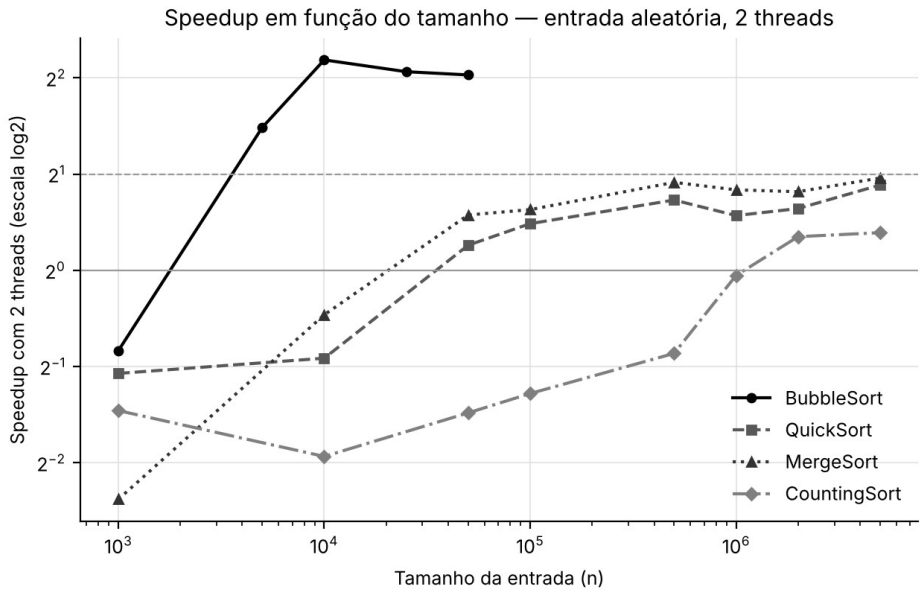
Bubble Sort. O Bubble Sort paralelo obteve speedup de 4,08 com 2 threads e 24,7 com 8 threads, valores muito acima do número de núcleos (speedup superlinear). Isso não significa que o paralelismo seja mais eficiente que o ideal: a versão paralela faz menos trabalho. Ao dividir o vetor em p blocos, o custo cai de n^2 para n^2/p, mais uma intercalação O(n log p) que é desprezível. Com 2 threads, o trabalho cai pela metade e os dois núcleos dividem o que restou, o que explica o fator próximo de 4. Para separar os dois efeitos, a seção 3.5 repete o experimento com um único núcleo. O resultado mostra uma lição importante: um speedup alto só pode ser atribuído ao paralelismo quando os dois lados da comparação fazem o mesmo trabalho.

Counting Sort. O Counting Sort foi o algoritmo que menos se beneficiou do paralelismo: speedup máximo de 1,31 com 2 threads em 5 milhões de elementos, e valores menores que 1 até 1 milhão. Há três razões. Primeiro, o algoritmo faz pouquíssimo cálculo por elemento (um incremento por valor lido), de modo que seu tempo é limitado pela largura de banda da memória, recurso compartilhado entre os núcleos. Segundo, a versão paralela precisa de um histograma de k posições por thread (4 MB cada), que deve ser alocado, zerado e depois somado; com 8 threads são 32 MB de histogramas, mais do que o próprio vetor de entrada para $n \leq 5$ milhões, o que explica por que P8 foi a pior configuração. Terceiro, são quatro fases separadas por barreiras, cada uma com custo de coordenação.

3.3 Efeito do tamanho da entrada

A Figura 5 mostra o speedup com 2 threads em função do tamanho, para a entrada aleatória. Em todos os algoritmos há um tamanho mínimo abaixo do qual a versão paralela é mais lenta que a serial (speedup menor que 1).

Figura 5 – Speedup com 2 threads em função do tamanho, entrada aleatória (linhas de referência em 1 e 2)



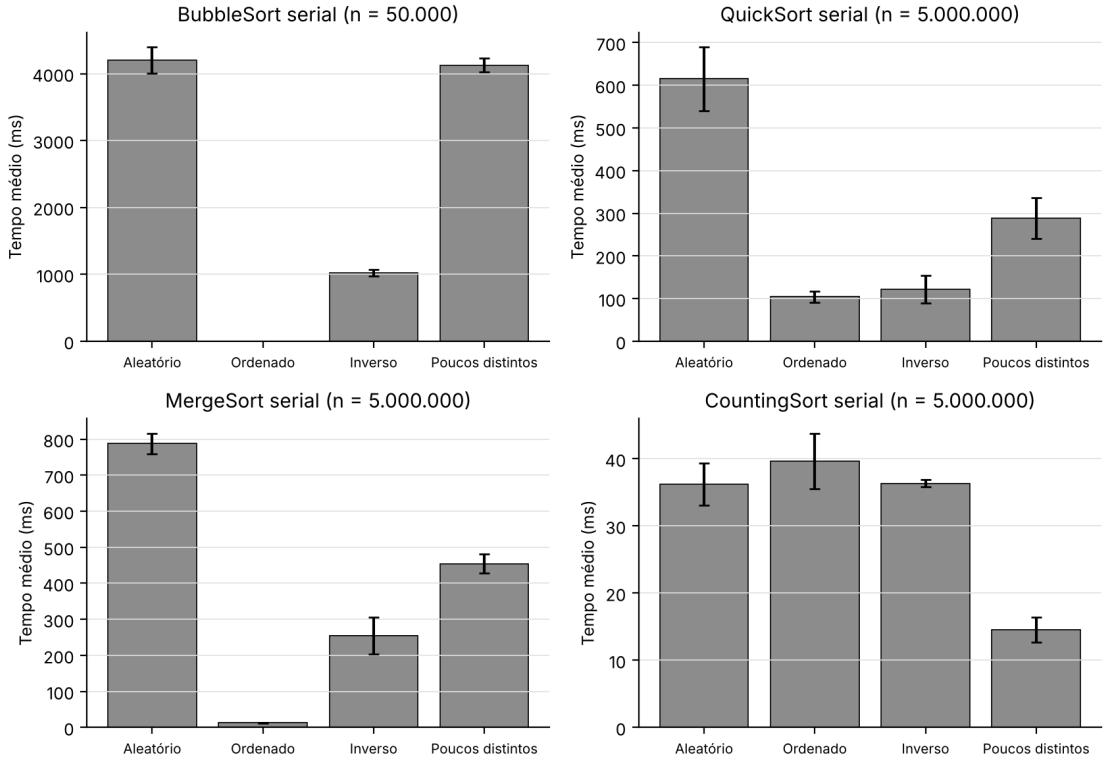
Fonte: elaborado pelo autor a partir dos arquivos CSV do experimento.

O menor tamanho com speedup maior que 1 foi de 50.000 elementos para o Quick Sort e o Merge Sort, 5.000 para o Bubble Sort e 2.000.000 para o Counting Sort. Abaixo desses valores, o tempo de criar o pool de threads, distribuir as tarefas e aguardar o término (da ordem de 0,3 a 1 ms nesta máquina) é maior do que o próprio tempo de ordenar. Para mil elementos, por exemplo, o Merge Sort serial levou 0,09 ms e o paralelo com 2 threads 0,45 ms, cinco vezes mais. O ganho cresce com o tamanho: a partir de 500 mil elementos, o speedup com 2 threads fica entre 1,5 e 1,9 para o Quick Sort e o Merge Sort. Na prática, implementações paralelas robustas usam exatamente esse critério: o `Arrays.parallelSort` do Java, por exemplo, recorre à ordenação serial quando o vetor tem menos de 8.192 elementos.

3.4 Efeito da natureza dos dados

A Figura 6 mostra o tempo serial de cada algoritmo para as quatro distribuições, no maior tamanho, e a Tabela 3 resume o tempo serial e o speedup com 2 threads.

Figura 6 – Tempo médio serial por natureza da entrada, no maior tamanho de cada algoritmo



Fonte: elaborado pelo autor a partir dos arquivos CSV do experimento.

Tabela 3 – Tempo serial (ms) / speedup com 2 threads, por natureza da entrada, no maior tamanho

Algoritmo	Aleatório	Ordenado	Inverso	Poucos distintos
BubbleSort	4.203,68 / 4,08	0,06 / 0,10	1.022,83 / 3,81	4.127,91 / 4,20
QuickSort	615,18 / 1,85	104,00 / 1,47	122,19 / 1,60	288,31 / 1,66
MergeSort	787,98 / 1,94	12,68 / 1,42	254,20 / 1,91	454,61 / 1,84
CountingSort	36,23 / 1,31	39,64 / 1,41	36,31 / 1,36	14,53 / 1,49

Fonte: elaborado pelo autor a partir dos arquivos CSV do experimento.

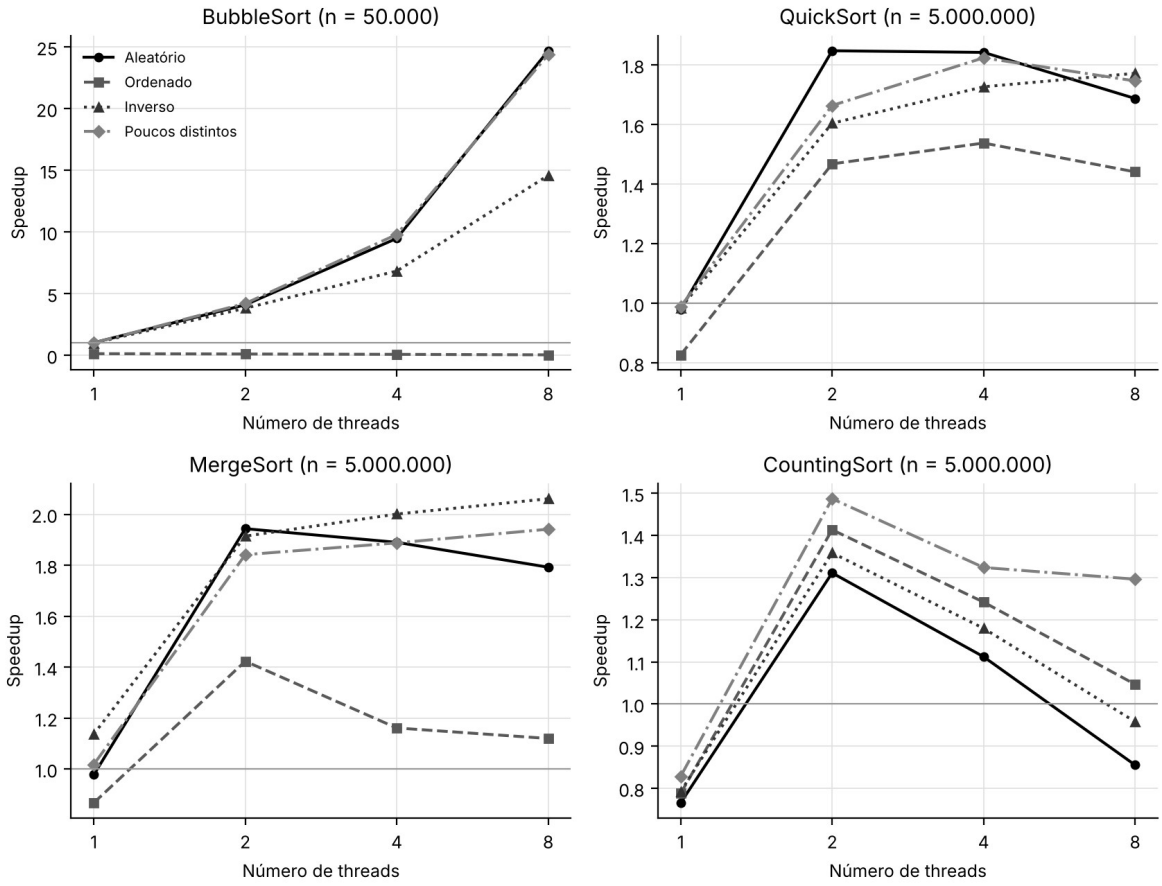
Bubble Sort. A entrada ordenada é o melhor caso: a primeira passada não faz trocas e a parada antecipada encerra o algoritmo em O(n) (0,06 ms para 50 mil elementos). Nesse caso a versão paralela é muito pior (speedup 0,10), pois o custo de criar threads é centenas de vezes maior que o trabalho a fazer. Um resultado menos óbvio é que a entrada inversa, teoricamente o pior caso (número máximo de trocas), foi quatro vezes mais rápida que a aleatória (1,0 s contra 4,2 s). A explicação está no previsor de desvios do processador: na entrada inversa, a comparação `v[i] > v[i+1]` tem resultado quase sempre igual, e o processador acerta a previsão; na aleatória, o resultado é imprevisível, e cada erro de previsão custa dezenas de ciclos. É um exemplo de como o hardware pode inverter conclusões tiradas apenas da contagem de operações.

Quick Sort e Merge Sort. Entradas ordenadas e inversas foram de 5 a 6 vezes mais rápidas que a aleatória no Quick Sort, pois a mediana de três escolhe o pivô ideal e o particionamento de Hoare quase não faz trocas (no caso inverso, a primeira

passada de trocas já deixa o vetor praticamente ordenado). No Merge Sort, a entrada ordenada foi 62 vezes mais rápida (12,7 ms), porque a otimização que pula a intercalação de metades já ordenadas torna o algoritmo praticamente linear. A entrada com poucos distintos também foi mais rápida que a aleatória nos dois algoritmos, novamente por causa de comparações mais previsíveis. Os maiores speedups ocorreram nas entradas com mais trabalho serial (aleatória e inversa); na ordenada, como o trabalho é pequeno, a parte fixa de coordenação pesa mais e o speedup caiu para 1,42 a 1,47.

Counting Sort. Como esperado, a ordem dos elementos quase não afeta o tempo (36 a 40 ms nas três distribuições com valores em [0, 1.000.000)). O que importa é a amplitude: com poucos distintos (k = 100), o vetor de contagem cabe na cache L1, e o tempo caiu para 14,5 ms. Foi também nesse caso que a versão paralela obteve o maior speedup (1,49), pois os histogramas locais passam a ser minúsculos e o custo de combiná-los desaparece.

Figura 7 – Speedup em função do número de threads, por natureza da entrada, no maior tamanho de cada algoritmo



Fonte: elaborado pelo autor a partir dos arquivos CSV do experimento.

A Figura 7 reúne o speedup por número de threads para as quatro distribuições. Para Quick, Merge e Counting Sort o padrão se repete: ganho de 2 threads em relação

a 1 e, depois, estabilidade ou queda, por falta de núcleos. Para o Bubble Sort, o speedup continua crescendo com 4 e 8 threads, pelo efeito algorítmico descrito na seção 3.2.

3.5 Efeito do número de núcleos

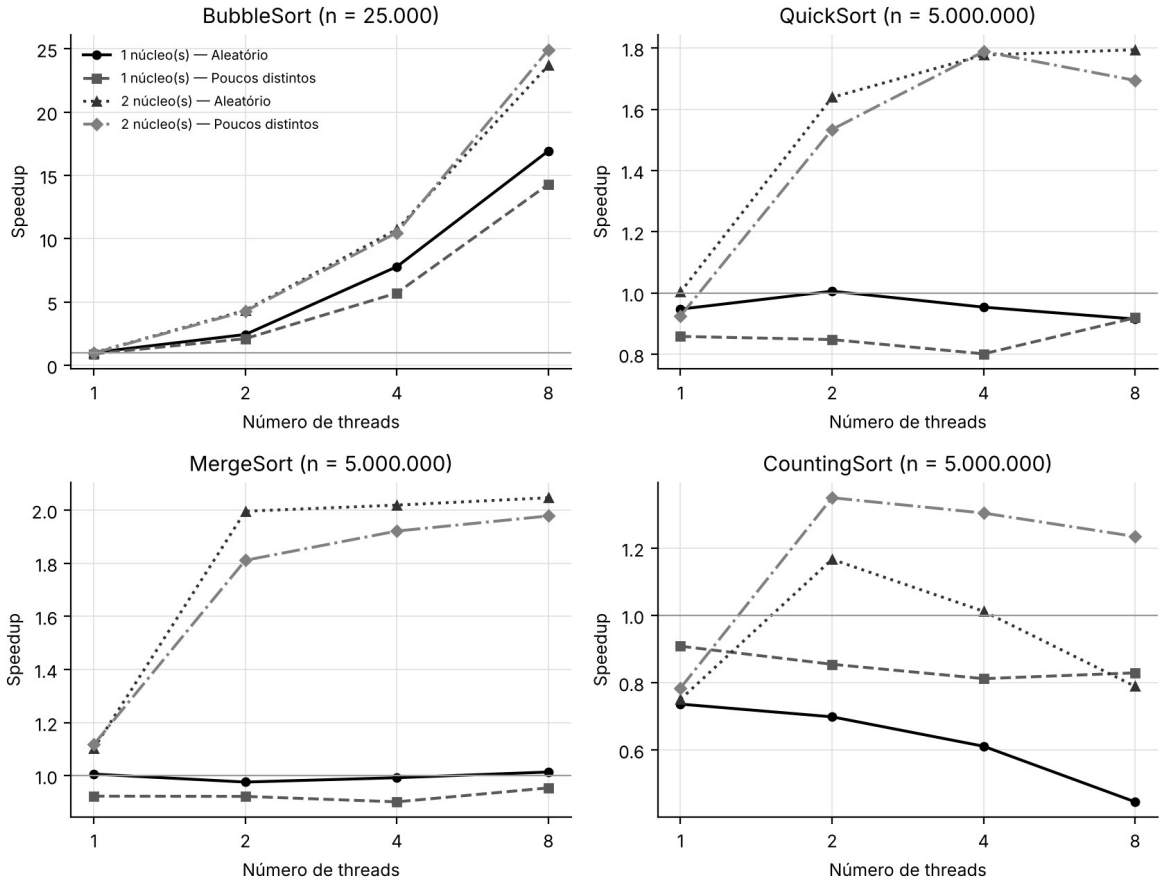
A Tabela 4 e a Figura 8 apresentam o experimento em que a JVM foi restrita a um e a dois núcleos.

Tabela 4 – Tempo serial (ms) e speedup com 2 e 8 threads, com 1 e 2 núcleos disponíveis

Algoritmo	Entrada	n	1 núcleo: serial	S(2)	S(8)	2 núcleos: serial	S(2)	S(8)
BubbleSort	Aleatório	25.000	978,04	2,44	16,93	955,64	4,35	23,69
BubbleSort	Poucos distintos	25.000	877,30	2,11	14,30	908,46	4,28	24,93
QuickSort	Aleatório	5.000.000	591,07	1,01	0,91	627,00	1,64	1,79
QuickSort	Poucos distintos	5.000.000	259,21	0,85	0,92	287,08	1,53	1,69
MergeSort	Aleatório	5.000.000	734,01	0,98	1,01	849,89	2,00	2,05
MergeSort	Poucos distintos	5.000.000	406,19	0,92	0,95	488,55	1,81	1,98
CountingSort	Aleatório	5.000.000	35,65	0,70	0,45	35,90	1,17	0,79
CountingSort	Poucos distintos	5.000.000	12,25	0,85	0,83	15,32	1,35	1,23

Fonte: elaborado pelo autor a partir dos arquivos CSV do experimento.

Figura 8 – Speedup em função do número de threads com 1 e 2 núcleos disponíveis



Fonte: elaborado pelo autor a partir dos arquivos CSV do experimento.

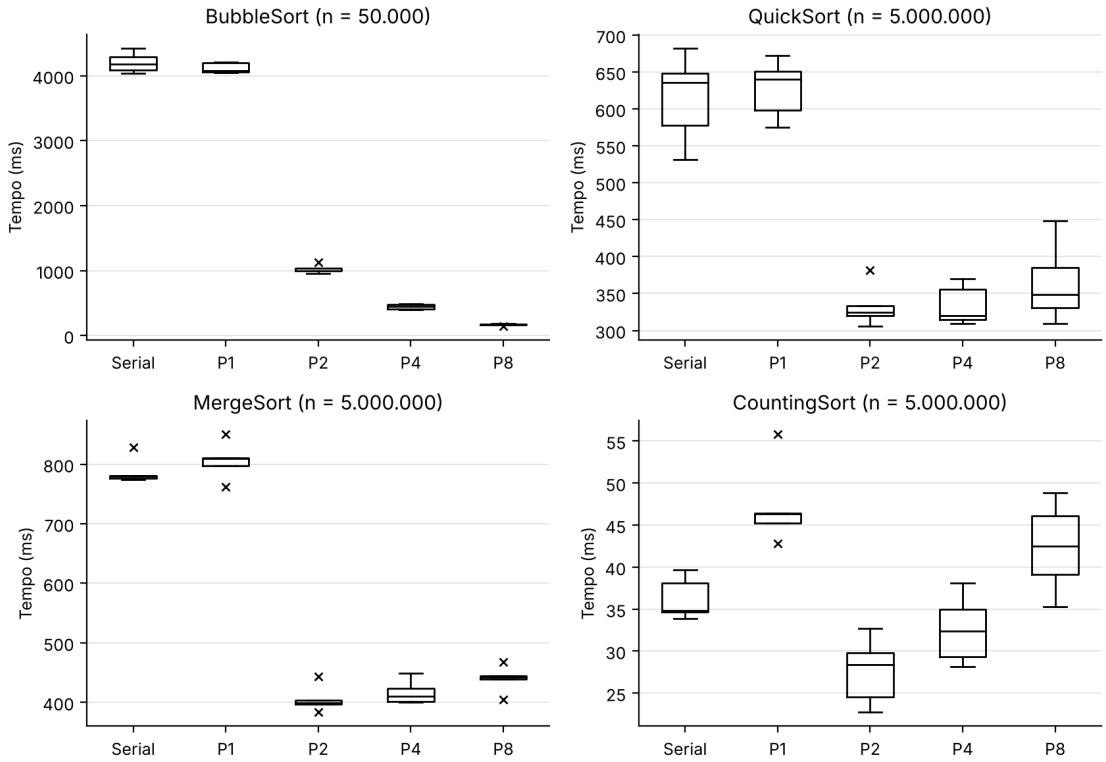
Com um único núcleo, o Quick Sort e o Merge Sort paralelos não tiveram ganho algum: o speedup ficou entre 0,80 e 1,01 para qualquer número de threads, pois as threads se revezam no mesmo núcleo e não há execução simultânea. Com dois núcleos, o speedup subiu para 1,4 a 2,0. O Counting Sort, com um núcleo, ficou sempre abaixo de 1 (0,45 com 8 threads na entrada aleatória), já que paga todos os custos extras da versão paralela sem nenhum benefício. Esse experimento confirma que o ganho dessas três versões vem exclusivamente da execução simultânea em núcleos distintos.

O Bubble Sort, por outro lado, obteve speedup de 16,9 com 8 threads mesmo em um único núcleo, o que prova que a maior parte do seu ganho é algorítmica (menos comparações e trocas), e não paralelismo. A redução de trabalho prevista pela análise é de 8 vezes; o restante provavelmente se deve à localidade de memória, já que um bloco de 25.000/8 ≈ 3.100 inteiros (12 KB) cabe inteiro na cache L1, enquanto o vetor completo (100 KB) não cabe. Comparando as duas colunas, o segundo núcleo multiplicou o speedup por 1,4 a 1,8 (de 2,44 para 4,35 com 2 threads e de 16,9 para 23,7 com 8), que é a contribuição efetiva do paralelismo.

3.6 Variabilidade das medições

A Figura 9 mostra a dispersão das cinco amostras no maior tamanho, e a Tabela 5 resume o coeficiente de variação (CV) de todas as configurações.

Figura 9 – Dispersão das cinco amostras (boxplot) no maior tamanho, entrada aleatória



Fonte: elaborado pelo autor a partir dos arquivos CSV do experimento.

Tabela 5 – Coeficiente de variação das configurações medidas

Grupo de configurações	Configurações	CV mediano (%)	Com CV ≤ 10% (%)
Todas as configurações	580	15,2	26
Média abaixo de 5 ms	241	20,4	15
Maiores tamanhos (Bubble ≥ 25 mil; demais ≥ 1 milhão)	220	11,1	45

Fonte: elaborado pelo autor a partir dos arquivos CSV do experimento.

A variabilidade foi maior nas medições curtas: configurações com média abaixo de 5 ms tiveram CV mediano de 20%, porque nessa escala qualquer interrupção do sistema operacional, troca de contexto ou pausa da JVM representa uma fração grande do tempo. Nos maiores tamanhos, o CV mediano caiu para 11%, ainda relativamente alto, o que é típico de máquinas virtuais compartilhadas, em que outros processos competem pelo mesmo hardware físico. Por isso todas as conclusões deste relatório foram tiradas a partir de diferenças grandes em relação ao IC 95% (por exemplo, o speedup de 1,94 do Merge Sort corresponde a uma diferença de 382 ms entre médias cujos IC são de cerca de 28 ms). Diferenças pequenas, como entre P2, P4 e P8 no Quick e no Merge Sort, ficam dentro da margem de erro e não devem ser

interpretadas como reais. Para reduzir a incerteza, recomenda-se aumentar o número de amostras (o framework aceita `--amostras=N`) e usar uma máquina dedicada.

3.7 Síntese: qual algoritmo para qual cenário

- **Inteiros em intervalo pequeno ou conhecido:** Counting Sort serial, que é o mais rápido e quase não depende da ordem dos dados. A versão paralela só compensa para entradas muito grandes, com ganho modesto.
- **Dados gerais e entradas grandes (centenas de milhares de elementos ou mais) em máquina multinúcleo:** Merge Sort ou Quick Sort paralelos com tantas threads quanto núcleos. O Merge Sort escalou de forma mais previsível; o Quick Sort não precisa de memória auxiliar.
- **Entradas pequenas (menos de cerca de 50 mil elementos):** versões seriais. O custo de coordenar threads supera o ganho.
- **Entradas já ordenadas ou quase ordenadas:** algoritmos que detectam ordem (Merge Sort com o teste de metades ordenadas, ou Bubble/Insertion Sort com parada antecipada) são praticamente lineares, e a paralelização é desnecessária.
- **Bubble Sort:** só faz sentido para fins didáticos. Sua versão paralela é mais rápida porque faz menos trabalho, mas continua ordens de grandeza mais lenta que os algoritmos O(n log n).

4 CONCLUSÃO

Este trabalho implementou quatro algoritmos de ordenação em Java, cada um com versão serial e paralela, e um framework de teste que variou o tamanho e a natureza dos dados, o número de threads e o número de núcleos, com cinco amostras por configuração e registro em CSV. A análise estatística das 3.600 medições permitiu identificar padrões claros de desempenho.

O primeiro é que o paralelismo tem um custo fixo, e só compensa quando o trabalho é grande o suficiente para amortizá-lo: com dois núcleos, Merge Sort e Quick Sort paralelos chegaram a speedups de 1,94 e 1,85 em cinco milhões de elementos, mas foram mais lentos que as versões seriais abaixo de cerca de 50 mil. O segundo é que o número de núcleos físicos é o limite efetivo: com um único núcleo não houve ganho, e usar 4 ou 8 threads em dois núcleos não melhorou o resultado obtido com 2. O terceiro é que algoritmos limitados pela memória, como o Counting Sort, escalam mal, porque a largura de banda é compartilhada entre os núcleos e a versão paralela adiciona tráfego de memória. O quarto é que speedups superlineares, como os do Bubble Sort paralelo, devem ser analisados com cuidado: o experimento com um núcleo mostrou que a maior parte do ganho vinha da redução de trabalho causada pela divisão em blocos, e não da execução simultânea.

A natureza dos dados também se mostrou decisiva: as entradas ordenadas tornaram vários algoritmos praticamente lineares, a amplitude dos valores determinou o desempenho do Counting Sort e efeitos de hardware, como a previsão de desvios, fizeram o pior caso teórico do Bubble Sort ser mais rápido que o caso aleatório.

Como limitações, destacam-se a máquina com apenas dois núcleos, que impediu observar a escalabilidade além de duas threads, e a variabilidade típica de um ambiente virtualizado. Como trabalhos futuros, sugerem-se: repetir os experimentos em uma máquina com 8 ou mais núcleos (basta executar os scripts do projeto); implementar a intercalação paralela no Merge Sort, para eliminar a última etapa sequencial; comparar as implementações com o `Arrays.parallelSort` da biblioteca padrão; e usar uma ferramenta de microbenchmark como o JMH para reduzir o ruído das medições curtas.

REFERÊNCIAS

AMDAHL, G. M. Validity of the single processor approach to achieving large scale computing capabilities. In: AFIPS SPRING JOINT COMPUTER CONFERENCE, 1967, Atlantic City. **Proceedings** [...]. New York: ACM, 1967. p. 483-485.

CORMEN, T. H.; LEISERSON, C. E.; RIVEST, R. L.; STEIN, C. **Algoritmos**: teoria e prática. 3. ed. Rio de Janeiro: Elsevier, 2012.

GEORGES, A.; BUYTAERT, D.; EECKHOUT, L. Statistically rigorous Java performance evaluation. In: ACM SIGPLAN CONFERENCE ON OBJECT-ORIENTED PROGRAMMING SYSTEMS, LANGUAGES AND APPLICATIONS (OOPSLA), 2007, Montreal. **Proceedings** [...]. New York: ACM, 2007. p. 57-76.

GOETZ, B. et al. **Java Concurrency in Practice**. Upper Saddle River: Addison-Wesley, 2006.

HOARE, C. A. R. Quicksort. **The Computer Journal**, v. 5, n. 1, p. 10-16, 1962.

KNUTH, D. E. **The Art of Computer Programming**, v. 3: Sorting and Searching. 2. ed. Reading: Addison-Wesley, 1998.

LEA, D. A Java fork/join framework. In: ACM CONFERENCE ON JAVA GRANDE, 2000, San Francisco. **Proceedings** [...]. New York: ACM, 2000. p. 36-43.

ORACLE. **Java Platform, Standard Edition 8 API Specification**: package java.util.concurrent. Disponível em: https://docs.oracle.com/javase/8/docs/api/java/util/concurrent/package-summary.html. Acesso em: 29 out. 2026.

PACHECO, P. S. **An Introduction to Parallel Programming**. Burlington: Morgan Kaufmann, 2011.

SEDGEWICK, R.; WAYNE, K. **Algorithms**. 4. ed. Upper Saddle River: Addison-Wesley, 2011.

ANEXO A – REPOSITÓRIO DO CÓDIGO

O código-fonte completo, os scripts de execução, os arquivos CSV com todos os resultados e o script de geração dos gráficos estão disponíveis no repositório:

https://github.com/Lucasvbc-Dev/computacao-paralela

Estrutura do repositório:

- `src/br/unifor/ordenacao/algoritmos` – interface `AlgoritmoOrdenacao` e as classes `BubbleSort`, `QuickSort`, `MergeSort`, `CountingSort` e `Util`;
- `src/br/unifor/ordenacao/dados` – `Distribuicao` e `GeradorDados`;
- `src/br/unifor/ordenacao/benchmark` – `Benchmark` (framework de teste), `Medicao` e `Estatisticas`;
- `src/br/unifor/ordenacao/grafico` – `VisualizadorGraficos` e `GraficoLinhas` (gráficos dinâmicos em Swing);
- `scripts` – compilação e execução (Linux e Windows); `analise` – geração dos gráficos; `resultados` – CSV e logs; `graficos` – figuras.

Para reproduzir: `./scripts/compilar.sh`, `./scripts/executar_benchmark.sh`, `./scripts/executar_experimento_nucleos.sh` e `./scripts/visualizar.sh` (ou os equivalentes `.bat` no Windows). Instruções completas no arquivo README.md.

ANEXO B – CÓDIGO-FONTE DAS IMPLEMENTAÇÕES

A seguir estão as classes dos algoritmos e do framework de teste. As classes de estatística e de visualização estão no repositório.

B.1 – AlgoritmoOrdenacao.java

```
package br.unifor.ordenacao.algoritmos;

/**
 * Contrato comum a todos os algoritmos do estudo.
 * Cada algoritmo oferece uma versão serial e uma versão paralela,
 * ambas ordenando o vetor in-place em ordem crescente.
 */
public interface AlgoritmoOrdenacao {

    /** Nome usado nos arquivos CSV e nos gráficos. */
    String nome();

    /** Ordena o vetor usando uma única thread. */
    void ordenarSerial(int[] v);

    /**
     * Ordena o vetor usando até {@code numThreads} threads de trabalho.
     * A criação e o encerramento do pool de threads fazem parte do custo medido.
     */
    void ordenarParalelo(int[] v, int numThreads);
}
```

B.2 – BubbleSort.java

```
package br.unifor.ordenacao.algoritmos;

import java.util.ArrayList;
import java.util.List;
import java.util.concurrent.ExecutorService;
import java.util.concurrent.Executors;
import java.util.concurrent.Future;

/**
 * Bubble Sort.
 *
 * Serial: versão clássica com parada antecipada (se uma passada não faz trocas, o vetor já está
ordenado).
 *
 * Paralelo: particionamento em blocos. O vetor é dividido em p blocos contíguos; cada thread ordena
 * o seu bloco com Bubble Sort; em seguida os blocos são intercalados em árvore (log2 p níveis),
 * com os pares de cada nível intercalados em paralelo. Como o custo do Bubble Sort é O(m^2),
 * ordenar p blocos de tamanho n/p custa p * (n/p)^2 = n^2/p, o que reduz o trabalho total
 * mesmo com uma única thread física disponível.
 */
public class BubbleSort implements AlgoritmoOrdenacao {

    @Override
    public String nome() {
        return "BubbleSort";
    }

    @Override
    public void ordenarSerial(int[] v) {
        bubble(v, 0, v.length);
    }
```

```
    /** Bubble Sort no intervalo [ini, fim). */
    static void bubble(int[] v, int ini, int fim) {
        for (int limite = fim - 1; limite > ini; limite--) {
            boolean trocou = false;
            for (int i = ini; i < limite; i++) {
                if (v[i] > v[i + 1]) {
                    int t = v[i];
                    v[i] = v[i + 1];
                    v[i + 1] = t;
                    trocou = true;
                }
            }
            if (!trocou) {
                return;
            }
        }
    }

    @Override
    public void ordenarParalelo(final int[] v, int numThreads) {
        if (v.length < 2) {
            return;
        }
        int blocos = Math.max(1, Math.min(numThreads, v.length));
        final int[] lim = Util.limitesBlocos(v.length, blocos);
        ExecutorService pool = Executors.newFixedThreadPool(numThreads);
        try {
            List<Future<?>> tarefas = new ArrayList<Future<?>>();
            for (int b = 0; b < blocos; b++) {
                final int ini = lim[b];
                final int fim = lim[b + 1];
                tarefas.add(pool.submit(new Runnable() {
                    public void run() {
                        bubble(v, ini, fim);
                    }
                }));
            }
            Util.aguardar(tarefas);
            Util.intercalarBlocos(v, lim, pool);
        } finally {
            pool.shutdown();
        }
    }
}
```

B.3 – QuickSort.java

```
package br.unifor.ordenacao.algoritmos;

import java.util.concurrent.ForkJoinPool;
import java.util.concurrent.RecursiveAction;

/**
 * Quick Sort com pivô pela mediana de três e particionamento de Hoare. A mediana de três evita o
 * pior caso O(n^2) em vetores ordenados/invertidos, e o esquema de Hoare (que para nos elementos
 * iguais ao pivô) mantém as partições equilibradas em vetores com muitos valores repetidos.
 * Subvetores com até {@value #CORTE_INSERCAO} elementos são finalizados com Insertion Sort.
 *
 * Paralelo: Fork/Join. Após particionar, as duas metades viram tarefas independentes
 * (não há dados compartilhados entre elas). Abaixo de {@value #LIMIAR_PARALELO} elementos,
 * a tarefa resolve o restante de forma serial para evitar sobrecarga de criação de tarefas.
 */
```

```
public class QuickSort implements AlgoritmoOrdenacao {

    static final int CORTE_INSERCAO = 32;
    static final int LIMIAR_PARALELO = 8192;

    @Override
    public String nome() {
        return "QuickSort";
    }

    @Override
    public void ordenarSerial(int[] v) {
        if (v.length > 1) {
            quick(v, 0, v.length - 1);
        }
    }

    /** Ordena o intervalo fechado [lo, hi]. Recursão no lado menor garante pilha O(log n). */
    static void quick(int[] v, int lo, int hi) {
        while (hi - lo > CORTE_INSERCAO) {
            int j = particionar(v, lo, hi);
            if (j - lo < hi - j) {
                quick(v, lo, j);
                lo = j + 1;
            } else {
                quick(v, j + 1, hi);
                hi = j;
            }
        }
        Util.insercao(v, lo, hi);
    }

    /**
     * Particionamento de Hoare. Ao final, todo elemento de v[lo..j] é menor ou igual a todo
     * elemento de v[j+1..hi]. Retorna j.
     */
    static int particionar(int[] v, int lo, int hi) {
        int pivo = medianaDeTres(v[lo], v[lo + (hi - lo) / 2], v[hi]);
        int i = lo - 1;
        int j = hi + 1;
        while (true) {
            do {
                i++;
            } while (v[i] < pivo);
            do {
                j--;
            } while (v[j] > pivo);
            if (i >= j) {
                return j;
            }
            Util.trocar(v, i, j);
        }
    }

    static int medianaDeTres(int a, int b, int c) {
        if (a < b) {
            if (b < c) {
                return b;
            }
            return (a < c) ? c : a;
        }
        if (a < c) {
            return a;
```

```
        }
        return (b < c) ? c : b;
    }

    @Override
    public void ordenarParalelo(int[] v, int numThreads) {
        if (v.length < 2) {
            return;
        }
        ForkJoinPool pool = new ForkJoinPool(numThreads);
        try {
            pool.invoke(new TarefaQuick(v, 0, v.length - 1));
        } finally {
            pool.shutdown();
        }
    }

    private static final class TarefaQuick extends RecursiveAction {
        private static final long serialVersionUID = 1L;
        private final int[] v;
        private final int lo;
        private final int hi;

        TarefaQuick(int[] v, int lo, int hi) {
            this.v = v;
            this.lo = lo;
            this.hi = hi;
        }

        @Override
        protected void compute() {
            if (hi - lo < LIMIAR_PARALELO) {
                if (hi > lo) {
                    quick(v, lo, hi);
                }
                return;
            }
            int j = particionar(v, lo, hi);
            invokeAll(new TarefaQuick(v, lo, j), new TarefaQuick(v, j + 1, hi));
        }
    }
}
```

B.4 – MergeSort.java

```
package br.unifor.ordenacao.algoritmos;

import java.util.concurrent.ForkJoinPool;
import java.util.concurrent.RecursiveAction;

/**
 * Merge Sort top-down com vetor auxiliar alocado uma única vez.
 * Subvetores com até {@value #CORTE_INSERCAO} elementos são ordenados com Insertion Sort, e a
 * intercalação é pulada quando as duas metades já estão em ordem (v[meio-1] <= v[meio]).
 *
 * Paralelo: Fork/Join. As duas metades são ordenadas como tarefas independentes e depois
 * intercaladas pela tarefa pai. A intercalação de cada nível é serial, então a última
 * intercalação (n elementos) limita o speedup máximo (Lei de Amdahl).
 */
public class MergeSort implements AlgoritmoOrdenacao {

    static final int CORTE_INSERCAO = 32;
    static final int LIMIAR_PARALELO = 8192;
```

```
@Override
public String nome() {
    return "MergeSort";
}

@Override
public void ordenarSerial(int[] v) {
    int[] aux = new int[v.length];
    merge(v, aux, 0, v.length);
}

/** Ordena o intervalo [ini, fim). */
static void merge(int[] v, int[] aux, int ini, int fim) {
    if (fim - ini <= CORTE_INSERCAO) {
        Util.insercao(v, ini, fim - 1);
        return;
    }
    int meio = (ini + fim) >>> 1;
    merge(v, aux, ini, meio);
    merge(v, aux, meio, fim);
    if (v[meio - 1] <= v[meio]) {
        return;
    }
    Util.intercalar(v, aux, ini, meio, fim);
}

@Override
public void ordenarParalelo(int[] v, int numThreads) {
    if (v.length < 2) {
        return;
    }
    int[] aux = new int[v.length];
    ForkJoinPool pool = new ForkJoinPool(numThreads);
    try {
        pool.invoke(new TarefaMerge(v, aux, 0, v.length));
    } finally {
        pool.shutdown();
    }
}

private static final class TarefaMerge extends RecursiveAction {
    private static final long serialVersionUID = 1L;
    private final int[] v;
    private final int[] aux;
    private final int ini;
    private final int fim;

    TarefaMerge(int[] v, int[] aux, int ini, int fim) {
        this.v = v;
        this.aux = aux;
        this.ini = ini;
        this.fim = fim;
    }

    @Override
    protected void compute() {
        if (fim - ini <= LIMIAR_PARALELO) {
            merge(v, aux, ini, fim);
            return;
        }
        int meio = (ini + fim) >>> 1;
        invokeAll(new TarefaMerge(v, aux, ini, meio), new TarefaMerge(v, aux, meio, fim));
```

```
            if (v[meio - 1] > v[meio]) {
                Util.intercalar(v, aux, ini, meio, fim);
            }
        }
    }
}
```

B.5 – CountingSort.java

```
package br.unifor.ordenacao.algoritmos;

import java.util.ArrayList;
import java.util.List;
import java.util.concurrent.ExecutorService;
import java.util.concurrent.Executors;
import java.util.concurrent.Future;

/**
 * Counting Sort para inteiros. Custo O(n + k), em que k = (max - min + 1) é a amplitude dos
valores.
 *
 * Paralelo, em quatro fases, todas divididas entre p threads:
 * 1. mínimo e máximo de cada bloco do vetor (redução final serial de p valores);
 * 2. histograma local de cada bloco (sem disputa: cada thread tem o seu próprio vetor de contagem);
 * 3. soma dos histogramas locais, com a faixa de valores [0, k) dividida entre as threads;
 * 4. reescrita do vetor: cada thread escreve os valores da sua faixa a partir de um deslocamento
 *    calculado por soma de prefixos (p valores, serial).
 */
public class CountingSort implements AlgoritmoOrdenacao {

    @Override
    public String nome() {
        return "CountingSort";
    }

    @Override
    public void ordenarSerial(int[] v) {
        if (v.length < 2) {
            return;
        }
        int min = v[0];
        int max = v[0];
        for (int x : v) {
            if (x < min) {
                min = x;
            } else if (x > max) {
                max = x;
            }
        }
        int k = max - min + 1;
        int[] cont = new int[k];
        for (int x : v) {
            cont[x - min]++;
        }
        int pos = 0;
        for (int i = 0; i < k; i++) {
            for (int c = cont[i]; c > 0; c--) {
                v[pos++] = i + min;
            }
        }
    }

    @Override
```

```
public void ordenarParalelo(final int[] v, int numThreads) {
    if (v.length < 2) {
        return;
    }
    final int p = Math.max(1, Math.min(numThreads, v.length));
    final int[] lim = Util.limitesBlocos(v.length, p);
    ExecutorService pool = Executors.newFixedThreadPool(numThreads);
    try {
        // Fase 1: mínimo e máximo por bloco
        final int[] mins = new int[p];
        final int[] maxs = new int[p];
        List<Future<?>> tarefas = new ArrayList<Future<?>>();
        for (int t = 0; t < p; t++) {
            final int id = t;
            tarefas.add(pool.submit(new Runnable() {
                public void run() {
                    int mn = v[lim[id]];
                    int mx = mn;
                    for (int i = lim[id] + 1; i < lim[id + 1]; i++) {
                        int x = v[i];
                        if (x < mn) {
                            mn = x;
                        } else if (x > mx) {
                            mx = x;
                        }
                    }
                    mins[id] = mn;
                    maxs[id] = mx;
                }
            }));
        }
        Util.aguardar(tarefas);
        int mn = mins[0];
        int mx = maxs[0];
        for (int t = 1; t < p; t++) {
            mn = Math.min(mn, mins[t]);
            mx = Math.max(mx, maxs[t]);
        }
        final int min = mn;
        final int k = mx - mn + 1;

        // Fase 2: histogramas locais
        final int[][] locais = new int[p][];
        tarefas.clear();
        for (int t = 0; t < p; t++) {
            final int id = t;
            tarefas.add(pool.submit(new Runnable() {
                public void run() {
                    int[] c = new int[k];
                    for (int i = lim[id]; i < lim[id + 1]; i++) {
                        c[v[i] - min]++;
                    }
                    locais[id] = c;
                }
            }));
        }
        Util.aguardar(tarefas);

        // Fase 3: soma dos histogramas, faixa de valores dividida entre as threads
        final int[] faixas = Util.limitesBlocos(k, p);
        final int[] total = new int[k];
        final long[] somaFaixa = new long[p];
        tarefas.clear();
```

```
            for (int t = 0; t < p; t++) {
                final int id = t;
                tarefas.add(pool.submit(new Runnable() {
                    public void run() {
                        long s = 0;
                        for (int i = faixas[id]; i < faixas[id + 1]; i++) {
                            int c = 0;
                            for (int[] local : locais) {
                                c += local[i];
                            }
                            total[i] = c;
                            s += c;
                        }
                        somaFaixa[id] = s;
                    }
                }));
            }
            Util.aguardar(tarefas);

            // Soma de prefixos sobre as p faixas: posição inicial de escrita de cada thread
            final int[] desloc = new int[p];
            long acum = 0;
            for (int t = 0; t < p; t++) {
                desloc[t] = (int) acum;
                acum += somaFaixa[t];
            }

            // Fase 4: reescrita do vetor ordenado
            tarefas.clear();
            for (int t = 0; t < p; t++) {
                final int id = t;
                tarefas.add(pool.submit(new Runnable() {
                    public void run() {
                        int pos = desloc[id];
                        for (int i = faixas[id]; i < faixas[id + 1]; i++) {
                            int valor = i + min;
                            for (int c = total[i]; c > 0; c--) {
                                v[pos++] = valor;
                            }
                        }
                    }
                }));
            }
            Util.aguardar(tarefas);
        } finally {
            pool.shutdown();
        }
    }
}
```

B.6 – Util.java

```
package br.unifor.ordenacao.algoritmos;

import java.util.ArrayList;
import java.util.List;
import java.util.concurrent.ExecutionException;
import java.util.concurrent.ExecutorService;
import java.util.concurrent.Future;

/** Rotinas auxiliares compartilhadas pelos algoritmos. */
final class Util {
```

```
private Util() {
}

/** Insertion Sort no intervalo fechado [lo, hi]. Usado para subvetores pequenos. */
static void insercao(int[] v, int lo, int hi) {
    for (int i = lo + 1; i <= hi; i++) {
        int chave = v[i];
        int j = i - 1;
        while (j >= lo && v[j] > chave) {
            v[j + 1] = v[j];
            j--;
        }
        v[j + 1] = chave;
    }
}

static void trocar(int[] v, int i, int j) {
    int t = v[i];
    v[i] = v[j];
    v[j] = t;
}

/**
 * Divide [0, n) em {@code partes} blocos contíguos de tamanho quase igual.
 * Retorna um vetor de limites com partes + 1 posições: o bloco k é [lim[k], lim[k+1]).
 */
static int[] limitesBlocos(int n, int partes) {
    int[] lim = new int[partes + 1];
    for (int k = 0; k <= partes; k++) {
        lim[k] = (int) ((long) n * k / partes);
    }
    return lim;
}

/** Intercala os trechos ordenados v[ini, meio) e v[meio, fim) usando aux no mesmo intervalo. */
static void intercalar(int[] v, int[] aux, int ini, int meio, int fim) {
    System.arraycopy(v, ini, aux, ini, fim - ini);
    int i = ini;
    int j = meio;
    int k = ini;
    while (i < meio && j < fim) {
        v[k++] = (aux[j] < aux[i]) ? aux[j++] : aux[i++];
    }
    while (i < meio) {
        v[k++] = aux[i++];
    }
    while (j < fim) {
        v[k++] = aux[j++];
    }
}

/**
 * Intercala, em árvore, blocos já ordenados definidos por {@code lim}.
 * A cada nível, os pares de blocos vizinhos são intercalados em paralelo no pool.
 */
static void intercalarBlocos(int[] v, int[] lim, ExecutorService pool) {
    int[] aux = new int[v.length];
    int[] limites = lim;
    while (limites.length > 2) {
        int blocos = limites.length - 1;
        List<Future<?>> tarefas = new ArrayList<Future<?>>();
        int novosBlocos = (blocos + 1) / 2;
        int[] novos = new int[novosBlocos + 1];
```

```
            for (int b = 0, nb = 0; b < blocos; b += 2, nb++) {
                novos[nb] = limites[b];
                if (b + 1 < blocos) {
                    final int ini = limites[b];
                    final int meio = limites[b + 1];
                    final int fim = limites[b + 2];
                    final int[] auxF = aux;
                    tarefas.add(pool.submit(new Runnable() {
                        public void run() {
                            intercalar(v, auxF, ini, meio, fim);
                        }
                    }));
                }
            }
            novos[novosBlocos] = limites[blocos];
            aguardar(tarefas);
            limites = novos;
        }
    }

    /** Aguarda todas as tarefas e propaga qualquer exceção ocorrida nas threads. */
    static void aguardar(List<Future<?>> tarefas) {
        for (Future<?> f : tarefas) {
            try {
                f.get();
            } catch (InterruptedException e) {
                Thread.currentThread().interrupt();
                throw new IllegalStateException("Execução interrompida", e);
            } catch (ExecutionException e) {
                throw new IllegalStateException("Falha em tarefa paralela", e.getCause());
            }
        }
    }
}
```

B.7 – GeradorDados.java

```
package br.unifor.ordenacao.dados;

import java.util.Arrays;
import java.util.Random;

/** Gera vetores de entrada reproduzíveis a partir de uma semente. */
public final class GeradorDados {

    public static final int VALOR_MAXIMO = 1_000_000;
    public static final int VALOR_MAXIMO_POUCOS_DISTINTOS = 100;

    private GeradorDados() {
    }

    public static int[] gerar(Distribuicao d, int n, long semente) {
        Random r = new Random(semente);
        int[] v = new int[n];
        switch (d) {
            case ALEATORIO:
                preencher(v, r, VALOR_MAXIMO);
                break;
            case ORDENADO:
                preencher(v, r, VALOR_MAXIMO);
                Arrays.sort(v);
                break;
            case INVERSO:
```

```
                preencher(v, r, VALOR_MAXIMO);
                Arrays.sort(v);
                inverter(v);
                break;
            case POUCOS_DISTINTOS:
                preencher(v, r, VALOR_MAXIMO_POUCOS_DISTINTOS);
                break;
            default:
                throw new IllegalArgumentException("Distribuição desconhecida: " + d);
        }
        return v;
    }

    private static void preencher(int[] v, Random r, int limite) {
        for (int i = 0; i < v.length; i++) {
            v[i] = r.nextInt(limite);
        }
    }

    private static void inverter(int[] v) {
        for (int i = 0, j = v.length - 1; i < j; i++, j--) {
            int t = v[i];
            v[i] = v[j];
            v[j] = t;
        }
    }
}
```

B.8 – Benchmark.java

```
package br.unifor.ordenacao.benchmark;

import br.unifor.ordenacao.algoritmos.AlgoritmoOrdenacao;
import br.unifor.ordenacao.algoritmos.BubbleSort;
import br.unifor.ordenacao.algoritmos.CountingSort;
import br.unifor.ordenacao.algoritmos.MergeSort;
import br.unifor.ordenacao.algoritmos.QuickSort;
import br.unifor.ordenacao.dados.Distribuicao;
import br.unifor.ordenacao.dados.GeradorDados;

import java.io.File;
import java.io.FileOutputStream;
import java.io.IOException;
import java.io.OutputStreamWriter;
import java.io.PrintWriter;
import java.nio.charset.StandardCharsets;
import java.util.ArrayList;
import java.util.Arrays;
import java.util.LinkedHashMap;
import java.util.List;
import java.util.Locale;
import java.util.Map;

/**
 * Framework de teste. Para cada algoritmo, tamanho, distribuição e amostra, gera um vetor de
entrada
 * (mesma semente para todas as configurações da amostra), executa a versão serial e a versão
paralela
 * com cada quantidade de threads, cronometra com System.nanoTime(), confere o resultado contra uma
 * ordenação de referência e grava uma linha no CSV de resultados brutos (gravação imediata, linha a
linha).
 * Ao final, gera o CSV de resumo estatístico.
 *
```

```
 * Parâmetros (todos opcionais, no formato --nome=valor):
 *   --algoritmos=BubbleSort,QuickSort,MergeSort,CountingSort
 *   --tamanhos=1000,10000,...            tamanhos para QuickSort, MergeSort e CountingSort
 *   --tamanhos-quadraticos=1000,5000,... tamanhos para BubbleSort (O(n^2))
 *   --distribuicoes=ALEATORIO,ORDENADO,INVERSO,POUCOS_DISTINTOS
 *   --threads=1,2,4,8
 *   --amostras=5
 *   --saida=resultados/brutos.csv
 *   --resumo=resultados/resumo.csv
 */
public final class Benchmark {

    private static final long SEMENTE_BASE = 20261029L;

    private Benchmark() {
    }

    public static void main(String[] args) throws IOException {
        Map<String, String> p = lerArgumentos(args);
        List<AlgoritmoOrdenacao> algoritmos = algoritmos(p.get("algoritmos"));
        int[] tamanhos = inteiros(p.get("tamanhos"));
        int[] tamanhosQuadraticos = inteiros(p.get("tamanhos-quadraticos"));
        int[] threads = inteiros(p.get("threads"));
        int amostras = Integer.parseInt(p.get("amostras"));
        List<Distribuicao> distribuicoes = new ArrayList<Distribuicao>();
        for (String d : p.get("distribuicoes").split(",")) {
            distribuicoes.add(Distribuicao.valueOf(d.trim()));
        }
        String saida = p.get("saida");
        String resumo = p.get("resumo");
        int nucleos = Runtime.getRuntime().availableProcessors();

        System.out.println("=== Benchmark de ordenação serial x paralela ===");
        System.out.println("Java " + System.getProperty("java.version") + " | núcleos disponíveis: "
+ nucleos
                + " | heap máx: " + (Runtime.getRuntime().maxMemory() / (1024 * 1024)) + " MB");
        System.out.println("Threads: " + Arrays.toString(threads) + " | amostras: " + amostras);

        aquecer(algoritmos, threads);

        File arq = new File(saida);
        if (arq.getParentFile() != null) {
            arq.getParentFile().mkdirs();
        }
        List<Medicao> todas = new ArrayList<Medicao>();
        PrintWriter csv = new PrintWriter(new OutputStreamWriter(new FileOutputStream(arq),
StandardCharsets.UTF_8));
        try {
            csv.println(Medicao.CABECALHO);
            csv.flush();
            long inicio = System.nanoTime();
            for (AlgoritmoOrdenacao alg : algoritmos) {
                int[] tams = (alg instanceof BubbleSort) ? tamanhosQuadraticos : tamanhos;
                for (int n : tams) {
                    for (Distribuicao d : distribuicoes) {
                        for (int a = 1; a <= amostras; a++) {
                            long semente = SEMENTE_BASE + 1000L * a + n;
                            int[] original = GeradorDados.gerar(d, n, semente);
                            int[] referencia = original.clone();
                            Arrays.sort(referencia);

                            todas.add(medir(csv, alg, "SERIAL", 1, nucleos, n, d, a, original,
referencia));
```

```
                            for (int t : threads) {
                                todas.add(medir(csv, alg, "PARALELO", t, nucleos, n, d, a, original,
referencia));
                            }
                        }
                        System.out.printf(Locale.US, "[%6.1fs] %-12s n=%-9d %-16s ok%n",
                                (System.nanoTime() - inicio) / 1e9, alg.nome(), n, d);
                    }
                }
            }
        } finally {
            csv.close();
        }

        for (Medicao m : todas) {
            if (!m.ordenadoOk) {
                System.err.println("ATENÇÃO: resultado incorreto em " + m.paraCsv());
            }
        }
        if (resumo != null && !resumo.isEmpty()) {
            Estatisticas.escreverResumo(Estatisticas.calcular(todas), resumo);
            System.out.println("Resumo estatístico: " + resumo);
        }
        System.out.println("Resultados brutos: " + saida + " (" + todas.size() + " medições)");
    }

    private static Medicao medir(PrintWriter csv, AlgoritmoOrdenacao alg, String versao, int
threads, int nucleos,
                                 int n, Distribuicao d, int amostra, int[] original, int[]
referencia) {
        int[] v = original.clone();
        System.gc();
        long t0 = System.nanoTime();
        if ("SERIAL".equals(versao)) {
            alg.ordenarSerial(v);
        } else {
            alg.ordenarParalelo(v, threads);
        }
        long t1 = System.nanoTime();
        boolean ok = Arrays.equals(v, referencia);
        Medicao m = new Medicao(alg.nome(), versao, threads, nucleos, n, d.name(), amostra, (t1 -
t0) / 1e6, ok);
        csv.println(m.paraCsv());
        csv.flush();
        return m;
    }

    /** Executa cada algoritmo algumas vezes antes das medições para que o compilador JIT otimize o
código. */
    private static void aquecer(List<AlgoritmoOrdenacao> algoritmos, int[] threads) {
        System.out.print("Aquecendo a JVM...");
        for (AlgoritmoOrdenacao alg : algoritmos) {
            int n = (alg instanceof BubbleSort) ? 3000 : 200_000;
            for (int r = 0; r < 5; r++) {
                int[] base = GeradorDados.gerar(Distribuicao.ALEATORIO, n, r);
                alg.ordenarSerial(base.clone());
                for (int t : threads) {
                    alg.ordenarParalelo(base.clone(), t);
                }
            }
        }
        System.out.println(" ok");
    }
```

```
    private static List<AlgoritmoOrdenacao> algoritmos(String nomes) {
        List<AlgoritmoOrdenacao> lista = new ArrayList<AlgoritmoOrdenacao>();
        for (String nome : nomes.split(",")) {
            String s = nome.trim();
            if (s.equalsIgnoreCase("BubbleSort")) {
                lista.add(new BubbleSort());
            } else if (s.equalsIgnoreCase("QuickSort")) {
                lista.add(new QuickSort());
            } else if (s.equalsIgnoreCase("MergeSort")) {
                lista.add(new MergeSort());
            } else if (s.equalsIgnoreCase("CountingSort")) {
                lista.add(new CountingSort());
            } else {
                throw new IllegalArgumentException("Algoritmo desconhecido: " + s);
            }
        }
        return lista;
    }

    private static int[] inteiros(String csv) {
        String[] partes = csv.split(",");
        int[] r = new int[partes.length];
        for (int i = 0; i < partes.length; i++) {
            r[i] = Integer.parseInt(partes[i].trim());
        }
        return r;
    }

    private static Map<String, String> lerArgumentos(String[] args) {
        Map<String, String> p = new LinkedHashMap<String, String>();
        p.put("algoritmos", "BubbleSort,QuickSort,MergeSort,CountingSort");
        p.put("tamanhos", "1000,10000,50000,100000,500000,1000000,2000000,5000000");
        p.put("tamanhos-quadraticos", "1000,5000,10000,25000,50000");
        p.put("distribuicoes", "ALEATORIO,ORDENADO,INVERSO,POUCOS_DISTINTOS");
        p.put("threads", "1,2,4,8");
        p.put("amostras", "5");
        p.put("saida", "resultados/brutos.csv");
        p.put("resumo", "resultados/resumo.csv");
        for (String a : args) {
            if (!a.startsWith("--") || !a.contains("=")) {
                throw new IllegalArgumentException("Argumento inválido: " + a + " (use --
nome=valor)");
            }
            int eq = a.indexOf('=');
            p.put(a.substring(2, eq), a.substring(eq + 1));
        }
        return p;
    }
}
```